

Anindya Biswas

UG-2, Physics

Roll No: 002420701050

Problem Set 3

Random Numbers:

1. Generate 100 integer random numbers that follow uniform distribution between (1,100). Then, find the mean

```
In [17]: import numpy as np
import random
np.random.seed(42)
distribution=np.random.randint(1,101, size=100)
print("Distribution:",distribution)
print("Mean:",np.mean(distribution))
```

Distribution: [52 93 15 72 61 21 83 87 75 75 88 100 24 3 22
53 2 88
30 38 2 64 60 21 33 76 58 22 89 49 91 59 42 92 60 80
15 62 62 47 62 51 55 64 3 51 7 21 73 39 18 4 89 60
14 9 90 53 2 84 92 60 71 44 8 47 35 78 81 36 50 4
 2 6 54 4 54 93 63 18 90 44 34 74 62 100 14 95 48 15
72 78 87 62 40 85 80 82 53 24]
Mean: 51.54

2. Find 100 random number that follow Gaussian distribution (normal distribution) with mean "zero" and standard deviation "1". Then calculate the mean and standard deviation of those numbers.

```
In [29]: np.random.seed(50)
mu=0. #mean
sigma=1. #standard deviation
gaussian=np.random.normal(mu,sigma,size=100)
print("Distribution: ",gaussian)
print("Mean:", np.mean(gaussian))
print("Standard deviation:", np.std(gaussian))
```

```

Distribution: [-1.56035211 -0.0309776 -0.62092842 -1.46458049 1.4119461
2 -0.47673214
-0.78046921 1.07026774 -1.2822926 -1.3274789 0.12633764 0.86219372
0.69673696 -0.33456518 -0.99752606 1.59890829 3.31407535 0.98777046
0.12386626 0.74278539 -0.39395585 0.14811582 -0.41223445 -0.16071506
0.13953147 0.28546937 -0.28126199 1.71090732 -0.14976664 0.69030672
1.09520951 1.3384087 -1.36898167 0.48642763 0.75352168 0.36346459
-0.31471048 1.37328117 -0.62441716 0.375754 -0.20041632 0.74303806
0.85736196 -1.50618929 -1.66635217 -0.2189948 -0.35885843 0.37852769
0.68421537 -1.16785607 -0.79321662 -0.0388354 2.70525499 -1.49138851
0.09597793 0.52468671 0.8156655 0.0515058 -0.16455536 0.27819942
0.0871147 0.03426837 0.74656892 -0.94363758 -0.24577667 1.10812096
0.03900789 -0.21326669 -0.89095461 -0.27064634 0.22224372 0.2518136
0.70792609 0.49398983 1.47100167 -0.58261947 2.06581949 1.08783389
0.80518904 -1.58788484 1.2308855 -2.37345338 -0.03111097 -3.80989038
-0.19904969 0.35043074 -0.05255078 -0.63428108 -0.36293111 -2.57091976
0.13751474 -0.71929619 0.72852289 1.62174549 1.52166225 -1.30677343
-0.8897683 0.00526774 0.86642487 -0.53553558]
Mean: 0.04004115872112115
Standard deviation: 1.0762778842618

```

3. Simulate a battle between two players rolling a 6-sided die. The player with the higher roll wins the round. Determine the winner after 10 rounds.

```

In [63]: val1=np.random.randint(1,7)
val2=np.random.randint(1,7)
print("Player1: ",val1)
print("Player2: ",val2)
if val1>val2:
    print("Player 1 wins")
elif val1==val2:
    print("It's a draw")
else:
    print("Player2 wins")
print("After 10 Rounds: ")
for i in range (1,11,1):
    val1=np.random.randint(1,7)
    val2=np.random.randint(1,7)
    print("Player1: ",val1)
    print("Player2: ",val2)
    if val1>val2:
        print("Player 1 wins")
    elif val1==val2:
        print("It's a draw")
    else:
        print("Player2 wins")

```

```

Player1: 1
Player2: 2
Player2 wins
After 10 Rounds:
Player1: 2
Player2: 4
Player2 wins

```

“read” from file and “write”, “add” in file

1. Read a text file (temperature.txt) containing daily temperature values (one per line). Calculate the average temperature and write the result to "summary.txt". File is attached.

```
In [97]: tempdat=np.loadtxt("temp.txt")
mean=np.mean(tempdat)
print("Mean:",mean)
file= open("summary.txt", 'w')
file1= open("temp.txt", 'a')

file.write(f"{mean}")
file1.write(f"\n{mean}")

file.close()
file1.close()
```

Mean: 16.72

2. Read a list of earthquake magnitudes from a file (earthquake_data.txt). Find the maximum, minimum, and average magnitude. File is attached.

```
In [101... richter = np.loadtxt("earthquake.txt")
print("Mean value of earthquake:",np.mean(richter))
print("Maximum magnitude:",np.max(richter))
print("Minimum magnitude:",np.min(richter))
```

Mean value of earthquake: 2.2699999999999996

Maximum magnitude: 6.3

Minimum magnitude: 1.0

3. Read a star catalogue file with names and brightness values (star.txt). Filter and save stars brighter than a threshold (e.g., magnitude < 0.1).

```
In [121... file=open("star.txt","r")
file1=open("brightstars.txt", "w")
for line in file:
    parts=line.strip().split(",")
    name=parts[0]
    mag=float(parts[1])
    if mag<0.1 :
        file1.write(f"{name},{mag}\n")
file.close()
file1.close()

file1=open("brightstars.txt", "r")
for i in file1.readlines():
    print(i)
```

Sirius, -1.46

Canopus, -0.74

Rigil Kentaurus, -0.27

Arcturus, -0.05

Vega, 0.03

Capella, 0.08

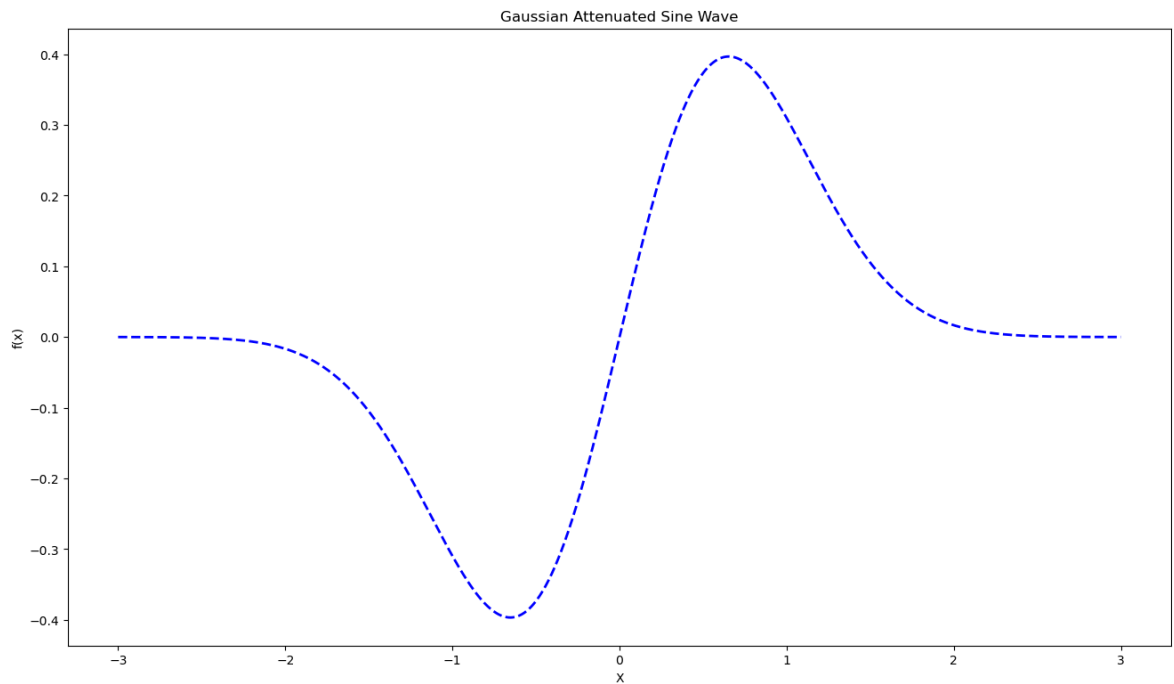
4. A chemist is monitoring the pH of a solution at different times and recording the values in a file (ph.txt). Each time a new pH measurement is taken, it is appended to the file without overwriting previous data.

```
In [136... phval=float(input("Type the value of pH"))
file=open("ph.txt","a")
file.write(f"{phval}\n")
file.close()
```

Plotting with Matplotlib

5. Write a Python program to plot the function: $f(x) = \sin(x) \cdot \exp(-x^2)$ for x values in the range $[-3, 3]$. Label the axes and add a title. Make the linewidth 2, linestyle '--', blue color. Save the plot in png format.

```
In [152... import matplotlib.pyplot as plt
import numpy as np
x=np.linspace(-3,3,1000)
f=np.sin(x)*np.exp(-x**2)
plt.figure(figsize=(16,9))
plt.plot(x,f,label= "Function plot",linewidth=2,color='blue',linestyle="--")
plt.xlabel("X")
plt.ylabel("f(x)")
plt.title("Gaussian Attenuated Sine Wave")
plt.savefig("plot1.png")
plt.show()
```



6. Plot the following functions on the same graph for x in the range $[-5, 5]$, give legends, title, x,y labels and save it in a single plot.

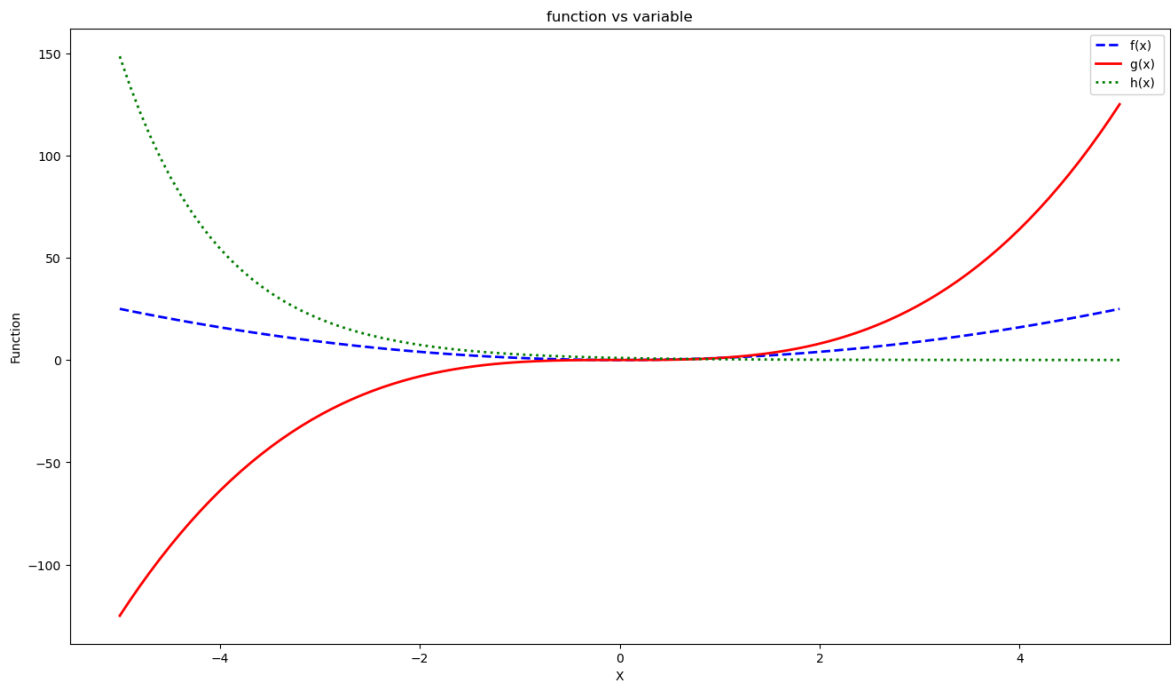
$f(x)=x^2$ (blue, dashed line)

$g(x)=x^3$ (red, solid line)

$h(x)=\exp(-x)$ (green, dotted line)

```
In [158... import matplotlib.pyplot as plt
import numpy as np
x= np.linspace(-5,5,100)
f=x**2
g=x**3
h=np.exp(-x)
plt.figure(figsize=(16,9))
plt.plot(x,f,label= "f(x) ",linewidth=2,color='blue',linestyle="--")
plt.plot(x,g,label= "g(x) ",linewidth=2,color='red',linestyle="-")
plt.plot(x,h,label= "h(x) ",linewidth=2,color='green',linestyle=":")

plt.xlabel("X")
plt.ylabel("Function")
plt.title("function vs variable")
plt.legend()
plt.savefig("plot2.png")
plt.show()
```



Write a Python program that reads the following data written in a text file (data.txt) containing two columns: time (s) and velocity (m/s). Then, plot the velocity vs. time graph.

t v

0 0

2 4.1

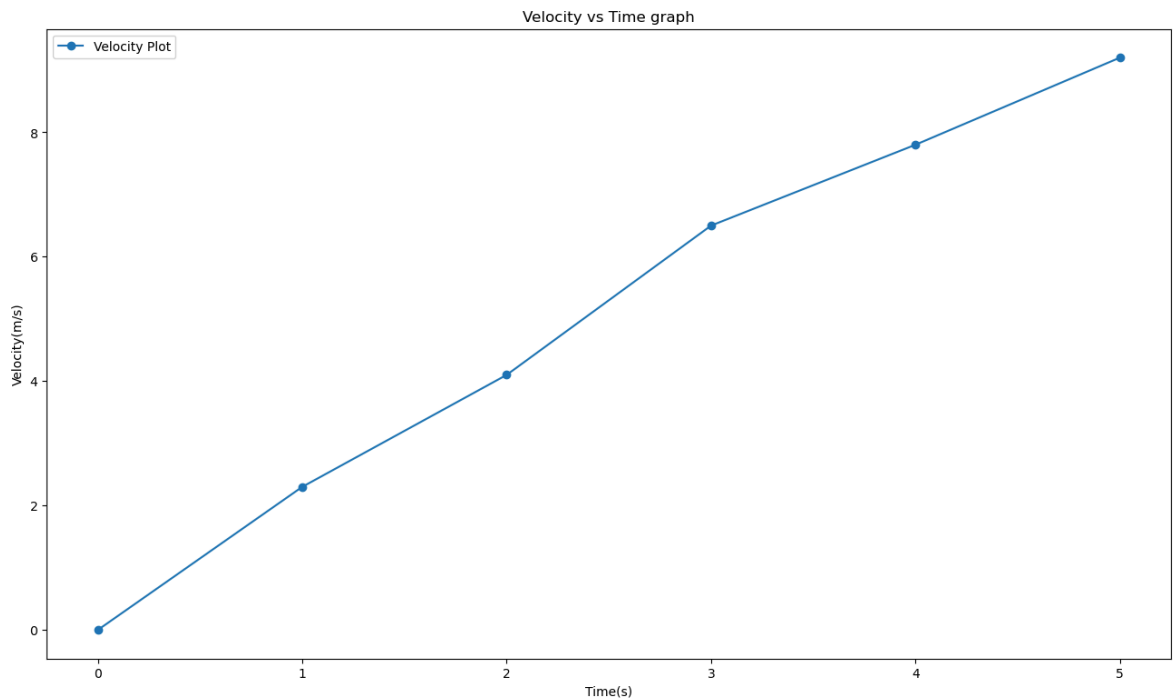
3 6.5

4 7.8

5 9.2

```
In [166... import matplotlib.pyplot as plt
import numpy as np
data=np.loadtxt("data.txt")

t=data[:,0]
v=data[:,1]
plt.figure(figsize=(16,9))
plt.plot(t,v,'o-',label='Velocity Plot')
plt.xlabel("Time(s)")
plt.ylabel("Velocity(m/s)")
plt.title("Velocity vs Time graph")
plt.legend()
plt.show()
```



8. (i) Generate a dataset that contains 1000 random Gaussian numbers with mean zero and standard deviation "1", store them in an array. Then, write a Python program to plot a histogram with 10 bins and save the plot.

(ii) Generate another dataset that contains 1000 random Gaussian numbers with mean 3 and standard deviation "2", store them in an array. Plot the histogram with 10 bins and save it separately, and compare with the first one.

```
In [186... import numpy as np
import matplotlib.pyplot as plt
np.random.seed(100)
dataset1=np.random.normal(0,1,size=1000)
dataset2=np.random.normal(3,2,size=1000)
plt.figure(figsize=(16,9))
plt.hist(dataset1, bins=10,color="blue")
plt.savefig("plot3.png")
plt.show()
plt.figure(figsize=(16,9))
plt.hist(dataset2,bins=10,color="orange")
plt.savefig("plot4.png")
plt.show()
plt.figure(figsize=(16,9))
plt.hist(dataset1,bins=10,color="blue")
plt.hist(dataset2,bins=10,color="orange")
plt.savefig("plot5.png")
plt.show()
```

